

Emulating Lua Execution through different Recurrent Neural Network Architectures: A Comparative Study in Example-Based Learning.

Richard Coric

University of Lincoln, School of Engineering and Physical Sciences
26414124@students.lincoln.ac.uk

Abstract. Deep learning, specifically language models, has become increasingly used in various software development tasks, such as chatbots for brainstorming and AI code completion in text editors — one aspect which is on the rise in the use of language models in compilers. This proposal introduces the main concepts of this technology and explains how it could be used within compilers to enable smoother language development resulting in more adaptive computing.

Keywords: Deep Learning · Compiler Development · Long Short-Term Memory (LSTM) · Recurrent Neural Network (RNN) · Gated Recurrent Unit (GRU).

1 Overview (465 / $\approx$ 500 Words)

1.1 Introduction

Schmidhuber first developed Long-Short Term Memory with the goal of creating a neural network architecture that improves on the old RNN architectures and focuses on eliminating its shortcomings, such as vanishing gradients (S. Hochreiter and J. Schmidhuber, 1997). Such a network is Memory-Based because it can decide what information it "remembers" and "forgets" via a gating mechanism.

There have since been many developments to improve RNNs for language modelling; one example is Transformers, an encoder-decoder model consisting of heads of attention and residual connections (Vaswani et al., 2023). One particular area of interest is using such technology in the domain of compiler development. We can use this technology to improve upon the current compiler architectures by using Neural Networks to learn part of, or most of, a compiler. Language Models have proven to be helpful in the area of compiler optimisation. Work done by Cummins et al. (2024) has shown that using an LLM (a local transformer model) can optimise code as part of a compiler. The Adaptive Neural Compiler demonstrates the use of neural networks in compilation by learning simple algorithms (Bunel et al., 2016).

The use of transformers for such a purpose has been widely explored; however, recent research by Feng et al. (2024) suggests that via minor adjustments (in the form of removing hidden layers) to LSTM and RNN models. These can achieve performance comparable to that of Transformer models, with the addition of parallelisability.

Example-based learning has been used in Machine Learning for decades; models tune their parameters based on a dataset of examples; this concept was further explored for programs by Zaremba et al. (2016), where they used an LSTM model along with a reinforcement learning approach which can from examples learn simple algorithms.

1.2 Rationale

Neural networks continue to revolutionise various computer science domains including compiler development with many works applying machine learning to compilers (H. Leather and C. Cummins, 2020; Cummins et al. 2024; Imada and Nakamura, 2008). This research aims to bridge this gap by investigating whether smaller models like RNNs, can still be useful in such a field. In addition to its being smaller models, the LSTM gating mechanism acts similarly to the idea of a virtual table in a compiler. In essence, a model could learn to "remember" variables and "forget" those that are out of scope. The success of this work could lead to more adaptive implementations of programming languages, with a potential application in domain-specific language development and compiler optimisation. Training a neural network for such a purpose could pose challenges due to its complex nature. However, Lua is an ideal candidate for this research due to its widespread use in embedded systems and its simplicity. This study pushes existing deep learning ideas towards language development and, in the future, towards more adaptive computer systems.

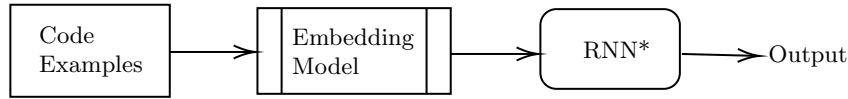


Fig. 1. An overview of the proposed architecture. Where *RNN** stands for any Recurrent Neural Network architecture.

2 Project Aims (205 / $\approx$ 200 words)

This project aims to develop a system, outlined in Figure 1 that can, given examples of code, predict the output of low (those that compute to a single output) and mid-to-high (those that compute to multiple outputs) complexity

programs, assuming they terminate. This is done to evaluate the effectiveness of the method. To achieve such a goal, this project will train, on examples, different RNNs architectures created using the PyTorch library for Python.

2.1 Objectives

1. Collect a suitable and substantial enough dataset of at least 300 examples of Lua code in order to train a machine-learning model.
2. Evaluate and appraise other methods of achieving an SLM compiler, which have already been researched.
3. Conduct a literature review to gain a critical understanding of work that has already been done in this field.
4. Develop an LSTM model, embedding model and lexer for parsing Lua, and then learn the model's embeddings.
5. Train different RNN models on embeddings created from examples of code and outputs, allowing it to find patterns and predict the output of code.
 - (a) Focusing on standardRNNs, LSTM, GRUs, and minimal versions of RNN and LSTM.
6. Test and evaluate the model based on new examples of Lua code, and perform relevant evaluation metrics such as Perplexity and Cross-Entropy.
7. Produce a report detailing the research and work.

3 Project Plan (232 / $\approx$ 300)

3.1 Time Planning

Figure 2 shows the Gantt chart which outlines the time plan for the duration of the research.

3.2 Risk Analysis

Table 1 highlights potential risks which may occur and the proposed Mitigation approach.

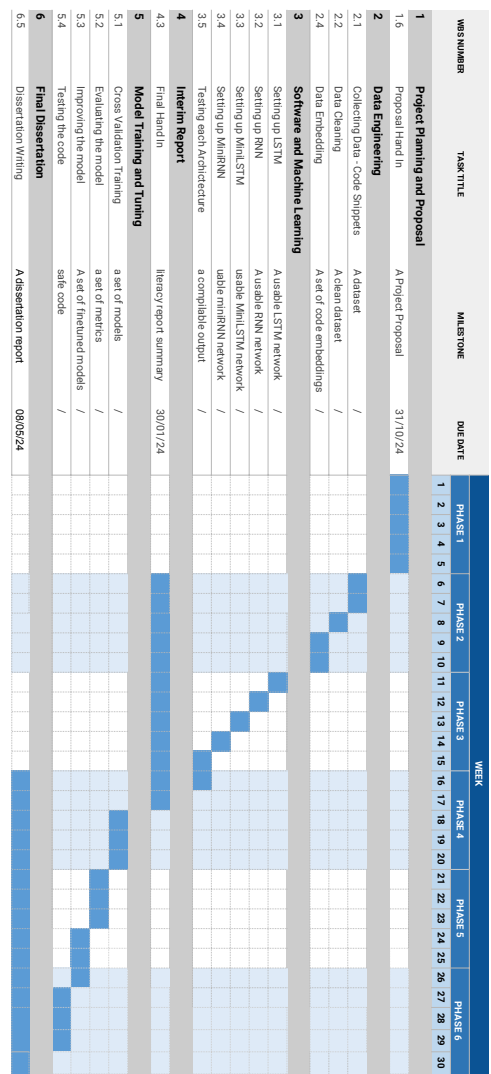


Fig. 2. Gantt Chart outlining the time plan for the dissertation project.

Table 1. Project Risk Analysis

Risk	Likelihood	Severity	Mitigation
Not finding enough training data or data that is of high enough quality	High	High	Data Augmentation - Transforming Existing Data or Zero-Shot Learning.
Model failing to fit and generalise, there is a risk LSTM is too simple or too complex	Mid	High	Hyperparameter tuning, early stopping, experimenting with the architecture, or cross-validation.
The chosen embedding method may not capture Lua semantics well enough.	Mid	High	Try different embedding methods, for example character level embeddings instead of using words.
The model could be computationally too expensive to train on a lot of data.	Low	Low	Using public GPUs, or optimising the model i.e paralling the architecture or simplifying the architecture.
It may be difficult to model the more intricate details of Lua.	Low	High	Using domain specific knowledge from Lua, using a more diverse dataset or more specific embeddings.
The chosen metrics may not full represent the model performance, giving a false picture.	Mid	Mid	Using and testing a variety of metrics.
May be difficult to compare against other methods.	Low	Mid	Using similar metrics or benchmark using a sandard dataset.

References

1. S. Hochreiter and J. Schmidhuber. (1997) Long Short-Term Memory. *Neural computation*, 9(8) 1735–1780 [accessed 18 October 2024]
2. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (August 2, 2023) Attention Is All You Need. In: Anonymous Available from <https://doi.org/10.48550/arXiv.1706.03762> [accessed 15 October 2024].
3. Feng, L., Tung, F., Ahmed, M.O., Bengio, Y. and Hajimirsadegh, H. (2024) Were RNNs All We Needed? Available from <https://doi.org/10.48550/arXiv.2410.01201> [accessed 20 October 2024].
4. Bunel, R., Desmaison, A., Kohli, P., Torr, P. and Mudigonda, P. (2016) Adaptive neural compilation. *Neural Information Processing Systems Foundation* [accessed 15 September 2024].
5. Zaremba, W., Mikolov, T., Joulin, A. and Fergus, R. (19 June 2016) Learning simple algorithms from examples. In: Anonymous *Proceedings of the 33rd International Conference on International Conference on Machine Learning*, June 19–24 2016. *JMLR.org*. 421–429 Available from <https://dl.acm.org/doi/10.5555/3045390.3045436> [accessed 20 September, 2024].
6. Cummins, C., Seeker, V., Grubisic, D., Roziere, B., Gehring, J., Synnaeve, G. and Leather, H. (2024) Meta Large Language Model Compiler: Foundation Models of Compiler Optimization. Available from <https://doi.org/10.48550/arXiv.2407.02524> [accessed Oct 20, 2024].
7. H. Leather and C. Cummins. (2020) Machine Learning in Compilers: Past, Present and Future. In: Anonymous *2020 Forum for Specification and Design Languages (FDL)* 1–8. Available from <https://doi.org/10.1109/FDL50818.2020.9232934> [accessed 24 October 2024].
8. Imada, K. and Nakamura, K. (2008) Towards machine learning of grammars and compilers of programming languages. In: Anonymous *Proceedings of the 2008th European Conference on Machine Learning and Knowledge Discovery in Databases - Volume Part II*, Antwerp, Belgium Berlin, Heidelberg: Springer-Verlag. 98–112. Available from: https://doi.org/10.1007/978-3-540-87481-2_7 [accessed 24 October 2024].